SMU Singapore Management University | School of Computing and Information Systems

Computational Thinking with Python

Jean Chen **James Koh**

{jeanchen, jameskohby}@smu.edu.sg

Sorting

Sorting

- Sorting is a very classic problem in computer science
- In both problem solving and data processing, sorting is a prerequisite for subsequent operations
- How to build sorting algorithms with loops?

Curr → Next?	Yes / No	At i^{th} position?	Yes / No
Cyclic patterns?	Yes / No	Terminating patterns?	Yes / No

© MITB, SMU

Insertion sort

- What is the “curr” pattern?
- What is the “next” pattern?

```
def insertion_sort(A):
    for i in range(1, len(A)):
        k = A[i]
        j = i - 1
        while j >= 0 and A[j] > k:
            A[j+1] = A[j]
            j -= 1
        A[j+1] = k
```


© MITB, SMU

Bubble sort

- What is the "curr" pattern?

- What is the "next" pattern?

start: 722 277 272 772 727

aft 1st pass: 277 272 722 727 772

aft 2nd pass:

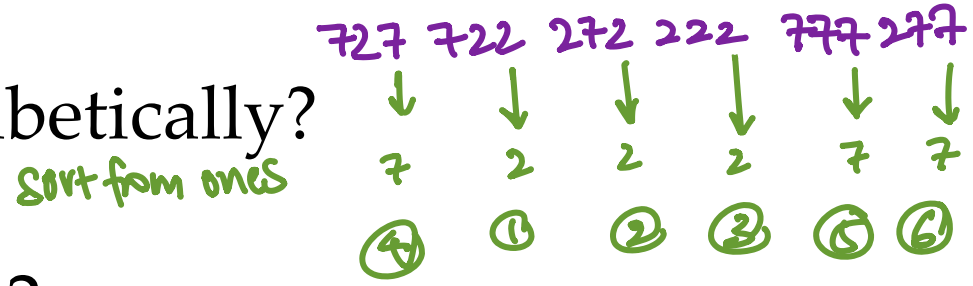
```
def bubble_sort(A):
    swapped = True
    while swapped:
        swapped = False
        for i in range(len(A)-1):
            if A[i] > A[i+1]:
                A[i], A[i+1] = A[i+1], A[i]
                swapped = True
```


© MITB, SMU

Radix Sort

sort from least sig to most sig
(for numbers, it is right to left)

- How to sort strings of equal length alphabetically?



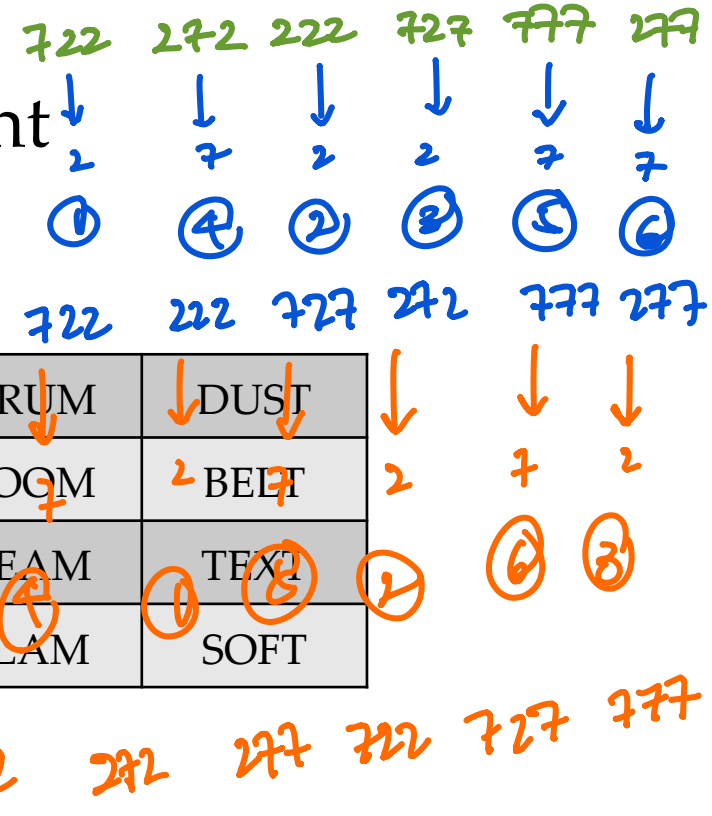
Option 1:

- Sort from left

BANK	DUST	SOCK	TASK
BELT	DRUM	SAND	TEXT
BOOM	DEED	SOFT	TEAM
BOLD	DARK	SLAM	TEND

Option 2:

- Sort from right



SAND	BANK	DRUM	DUST
DEED	SOCK	BOOM	BELT
BOLD	TASK	TEAM	TEXT
TEND	DARK	SLAM	SOFT

© MITB, SMU

Example Run

toy1=radix_sort_by(toy,3) toy2=radix_sort_by(toy,2) toy3=radix_sort_by(toy,1) toy4=radix_sort_by(toy,0)

Sort by the last letter:

SAND
DEED
BOLD
TEND
BANK
SOCK
TASK
DARK
DRUM
BOOM
TEAM
SLAM
DUST
BELT
TEXT
SOFT

Sort by the third letter:

TEAM
SLAM
SOCK
DEED
SOFT
BOLD
BELT
SAND
TEND
BANK
BOOM
DARK
TASK
DUST
DRUM
TEXT

Sort by the second letter:

SAND
BANK
DARK
TASK
TEAM
DEED
BELT
TEND
TEXT
SLAM
SOCK
SOFT
BOLD
BOOM
DRUM
DUST

Sort by the first letter:

BANK
BELT
BOLD
BOOM
DARK
DEED
DRUM
DUST
SAND
SLAM
SOCK
SOFT
TASK
TEAM
TEND
TEXT

Extension of Radix Sort

- Can radix sort be extended?
- Stable sorting: preserves the order of elements with equal keys.
- If a sorting algorithm is stable, and you're sorting by multiple keys, you should sort starting from the least significant (or minority) key, then proceed to the most significant (or majority) key.
- Example: sort customers by `(age <desc>, enqueue_time <asc>)`

Example:

- Consider the substrings starting from the second character onward, already sorted: $s_1 < s_2 < s_3$
- Now let us sort by the first character
- What should be the correct order?

First Char	already sorted
Q	-- s_1 --
P	-- s_2 --
Q	-- s_3 --

© MITB, SMU

Radix Sort


```
def radix_sort(s):
    # s: a list of strings to be sorted
    k = len(s[0])
    for i in range(k-1, -1, -1):
        s = radix_sort_by(s, i)
    return s
```

```
def radix_sort_by(s, i):
    n, d, res = len(s), {}, []
    for j in range(n):
        if s[j][i] in d:
            d[s[j][i]].append(s[j])
        else:
            d[s[j][i]] = [s[j]]
    for k in sorted(d.keys()):
        res.extend(d[k])
    return res
```


© MITB, SMU

Bit Operations

bin()

- `bin` converts a specified integer number to its binary representation
  - `bin(integer)`
  - `returns with prefix '0b'`
  - `'0b' means it is a base-2 number`

```
>>> bin(1)
'0b1'
>>> bin(2)
'0b10'
>>> bin(3)
'0b11'
```

- Exercise: *bin()*

© MITB, SMU

int()

- `int` converts the specified value into an integer number
  - `int(value, base = 10)`
  - `value`: A number or a string that can be converted into an integer number
  - `base`: A number representing the number format.

```
>>> int(5.9)
5
>>> int("11", 2)
3
>>> int("0b11", 2)
3
>>> int("a1b3", 16)
41395
```

$1 \times 2^0 + 1 \times 2^1$

- Exercise: *int()*

$3 \times 16^0 + 11 \times 16^1 + 1 \times 16^2 + 10 \times 16^3$

© MITB, SMU

Wooclap!

What is printed by the below code?

```
print(int(bin(11), 2))
```

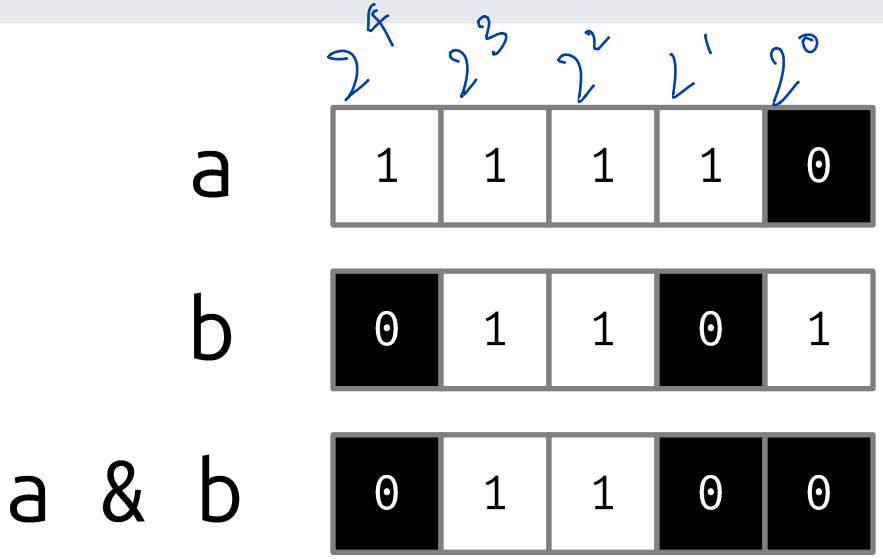
convert back to base 10 — convert to bin

a. 11
b. 3
c. 2
d. An error is observed

Bitwise Logical Operators

Operator	Description	Example
AND: &	**Returns a one only when both bits are switched on.**	**a & b -> 12**
OR: |	Returns a one if at least one of them is switched on.	a | b -> 31
XOR: ^	Returns a one only if every bit pair has opposing bit values.	a ^ b -> 19
NOT: ~	Turns zeros into ones and ones into zeros.	~a -> -31

```
a, b = 30, 13
>>> bin(a)
'0b11110'
>>> bin(b)
'0b1101'
```



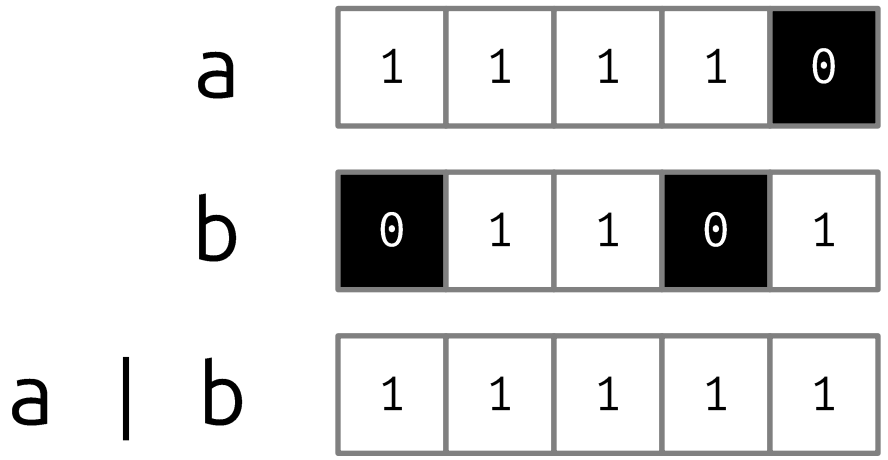
$$0 \times 2^0 + 0 \times 2^1 + 1 \times 2^2 + 1 \times 2^3 + 0 \times 2^4 = 12$$

© MITB, SMU

Bitwise Logical Operators

Operator	Description	Example
AND: &	Returns a one only when both bits are switched on.	a & b -> 12
OR: |	**Returns a one if at least one of them is switched on.**	**a | b -> 31**
XOR: ^	Returns a one only if every bit pair has opposing bit values.	a ^ b -> 19
NOT: ~	Turns zeros into ones and ones into zeros.	~a -> -31

```
a, b = 30, 13
>>> bin(a)
'0b11110'
>>> bin(b)
'0b1101'
```



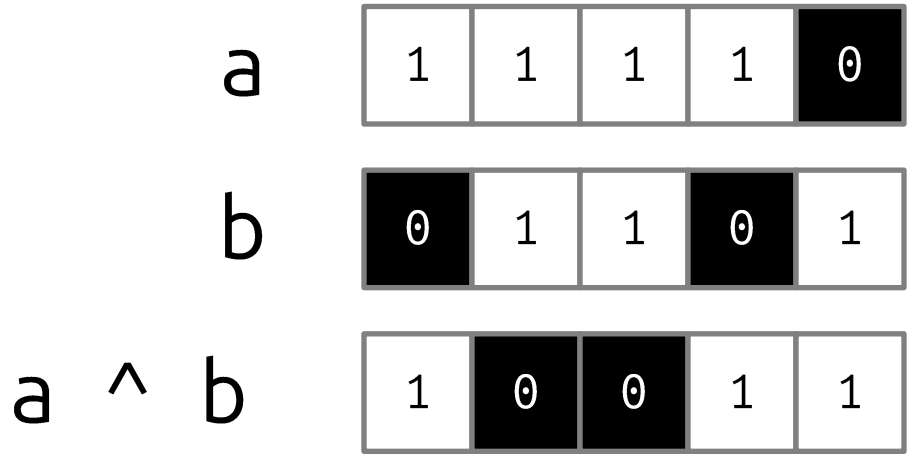
$$1 \times 2^0 + 1 \times 2^1 + 1 \times 2^2 + 1 \times 2^3 + 1 \times 2^4 = 31$$

© MITB, SMU

Bitwise Logical Operators

Operator	Description	Example
AND: &	Returns a one only when both bits are switched on.	a & b -> 12
OR: |	Returns a one if at least one of them is switched on.	a | b -> 31
XOR: ^	**Returns a one only if every bit pair has opposing bit values.**	**a ^ b -> 19**
NOT: ~	Turns zeros into ones and ones into zeros.	~a -> -31

```
a, b = 30, 13
>>> bin(a)
'0b11110'
>>> bin(b)
'0b1101'
```



$$1 \times 2^0 + 1 \times 2^1 + 0 \times 2^2 + 0 \times 2^3 + 1 \times 2^4 = 19$$

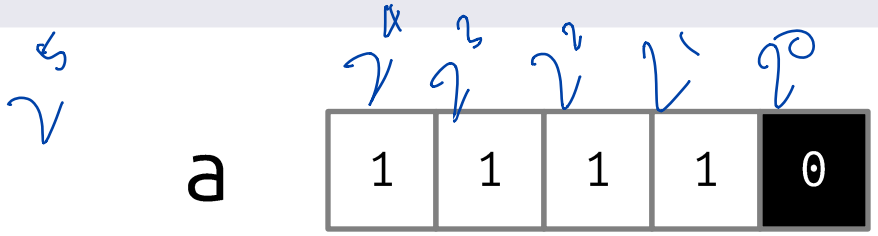
© MITB, SMU

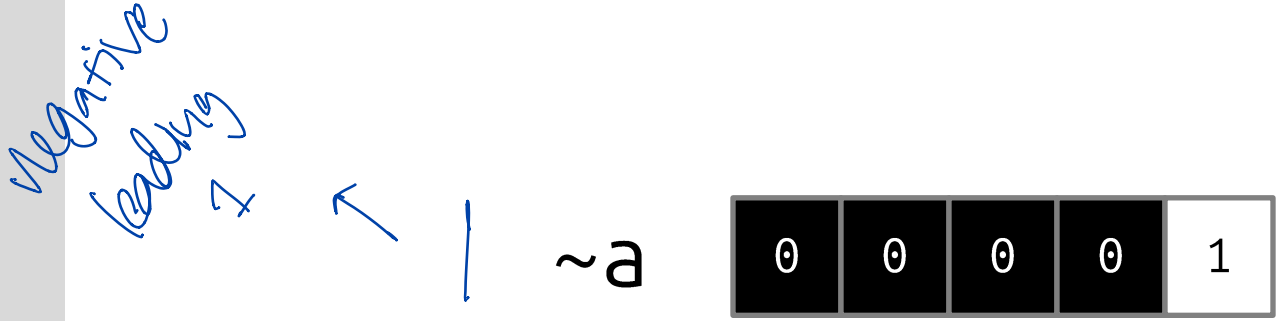
Bitwise Logical Operators

Operator	Description	Example
AND: &	Returns a one only when both bits are switched on.	a & b -> 12
OR: |	Returns a one if at least one of them is switched on.	a | b -> 31
XOR: ^	Returns a one only if every bit pair has opposing bit values.	a ^ b -> 19
NOT: ~	**Turns zeros into ones and ones into zeros.**	**~a -> -31**

$\sim x = -(x+1)$

```
a = 30
>>> bin(a)
'0b11110'
>>> ~a
-31
```





$$1 \times 2^0 + 0 \times 2^1 + 0 \times 2^2 + 0 \times 2^3 + 0 \times 2^4 = 1$$

© MITB, SMU

Why Study Bitwise Logical Operators?

Wooclap!

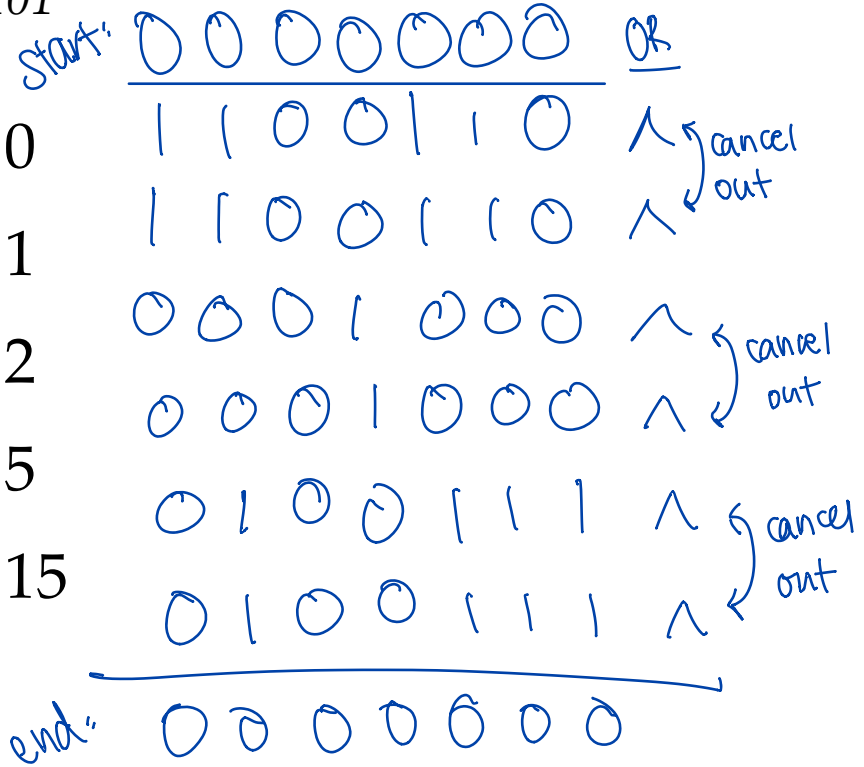
Consider the below function. What is the output of the print statement?

Hint: '0b1101' ^ '0b1101' = '0b0000', and '0b1101' ^ '0b0000' = '0b1101'

```
def func(input_list):
    unique = 0
    for num in input_list:
        unique = unique ^ num
    return unique

arr = [1, 2, 2, 1, 3, 5, 4, 3, 4]
print(func(arr))
```

a. 0
b. 1
c. 2
d. 5
e. 15



Wooclap! Explanation

- The previous question is an interesting application of bitwise logical operators.
- Notice that the XOR operator can be used to find the **single non-duplicate element** in a list where every other element appears twice.
- Naturally, this can be expanded to other applications, e.g., check if all the numbers in a list are duplicates.

```
def func(input_list):
    unique = 0
    for num in input_list:
        unique = unique ^ num
    return unique

arr = [1, 2, 2, 1, 3, 5, 4, 3, 4]
print(func(arr))
```

Wooclap!

What is the output of the below print statement?

Hint: bin(10) = '0b1010' (or '0b01010') and bin(20) = '0b10100' and 10 ^ 20 = 30

```
def func(a, b):
    a = a ^ b
    b = a ^ b
    a = a ^ b
    return a, b


x, y = 10, 20
x, y = func(x, y)
print(x, y)
```

a. 10 20

b. 20 10

c. 10 30

d. 30 20

e. 30 30

```
def func(a, b):
    a_new = a ^ b = (a_0 ^ b_0)
    b_new = a_new ^ b_0 = (a_0 ^ b_0) ^ b_0 = a_0
    a = a_new ^ b_new = (a_0 ^ b_0) ^ (a_0 ^ b_0) ^ b_0 = b_0
    return a, b

x, y = 10, 20 → reverse it
x, y = func(x, y)
print(x, y) # 20, 10
```

Wooclap! Explanation

- Here, the XOR operator can be used to swap two numbers **without using extra memory**!

```
def func(a, b):
    a = a ^ b
    b = a ^ b
    a = a ^ b
    return a, b

x, y = 10, 20
x, y = func(x, y)
print(x, y)
```

Integers

Optional

- Positive integers are represented in binary numeral system
- Bits: a blend from "binary digits"
- 4 bits can represent 16 distinct integers
  32 bits can represent 2^{32} distinct integers

```
>>> from bitstring import BitArray
>>> i1 = BitArray(int = 5, length = 4)
>>> i1.bin
'0101'
>>> i2 = BitArray(int = -1, length = 4)
>>> i2.bin
'1111'
```

Binary representation	When representing only non-negative integers (a.k.a. unsigned integers) with 4 bits															
	0000	0001	0010	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100	1101	1110	1111
Integers	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

- How are negative integers represented in Python?

© MITB, SMU

Binary representation of integers

Optional

School of Computing and Information Systems

Binary representation	When representing only non-negative integers (a.k.a. unsigned integers) with 4 bits															
	0000	0001	0010	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100	1101	1110	1111
Integers	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

Reserve this part for negative numbers

- We need to scarify half of the spaces for negative integers
- Keep the lower half unchanged to minimize interruptions
- the first bit is the sign bit
  - 0 for non-negative (0 or positive) numbers
  - 1 for negative numbers

Button: optional reading for the next slide
Do you know how negative numbers are represented?

© MITB, SMU

Binary representation of integers

Optional

Binary representation	Option 1							
	1000	1001	1010	1011	1100	1101	1110	1111
Integers	-1	-2	-3	-4	-5	-6	-7	-8

- Option 1: 1000 is chosen as -1 and 1111 is chosen as -8
  - However,

In binary	1000	+	1	=	1001
Integers	(-1)	+	1	≠	(-2)

- Option 2: 1000 is chosen as -8 and 1111 is chosen as -1

This is a better option

Binary representation	Option 2							
	1000	1001	1010	1011	1100	1101	1110	1111
Integers	-8	-7	-6	-5	-4	-3	-2	-1

© MITB, SMU

Binary representation of integers

Optional

Binary representation	When representing only non-negative integers (a.k.a. unsigned integers) with 4 bits															
	0000	0001	0010	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100	1101	1110	1111
Integers	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

- If there are n bits,
  - integers 0 to 2^{n-1}-1 are represented using 0 as the first bit, and
  - integers from -2^{n-1} to -1 are represented by 1 as the first bit
- the first bit is the sign bit

Binary representation	When representing both positive and negative integers (a.k.a. signed integers) with 4 bits															
	1000	1001	1010	1011	1100	1101	1110	1111	0000	0001	0010	0011	0100	0101	0110	0111
Integers	-8	-7	-6	-5	-4	-3	-2	-1	0	1	2	3	4	5	6	7

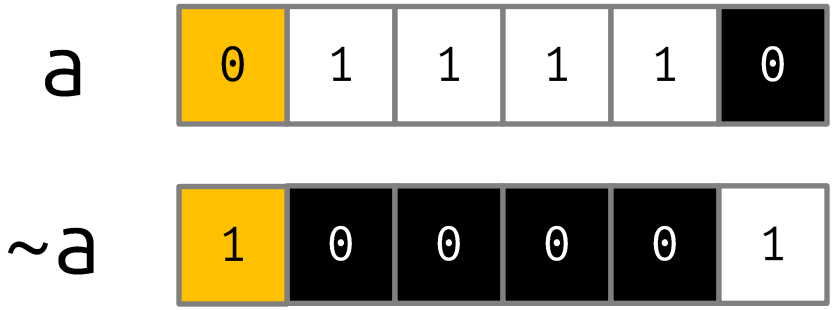
© MITB, SMU

Bitwise Logical Operators

Operator	Description	Example
AND: &	Returns a one only when both bits are switched on.	a & b -> 12
OR: |	Returns a one if at least one of them is switched on.	a | b -> 31
XOR: ^	Returns a one only if every bit pair has opposing bit values.	a ^ b -> 19
NOT: ~	**Turns zeros into ones and ones into zeros.**	**~a -> -31**

$\sim x = -(x+1)$

```
a = 30
>>> bin(a)
'0b11110'
>>> ~a
-31
```



$-1 \times 2^5 + 1 = -31$

- Exercise: *Bitwise Logical Operators*

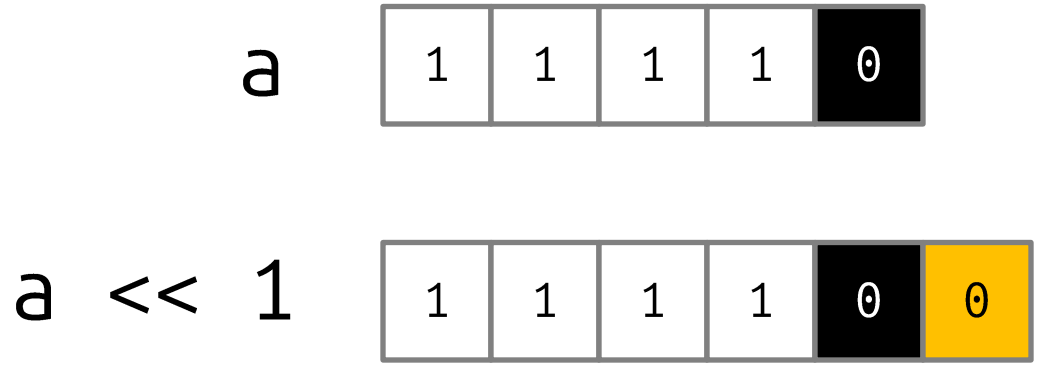
© MITB, SMU

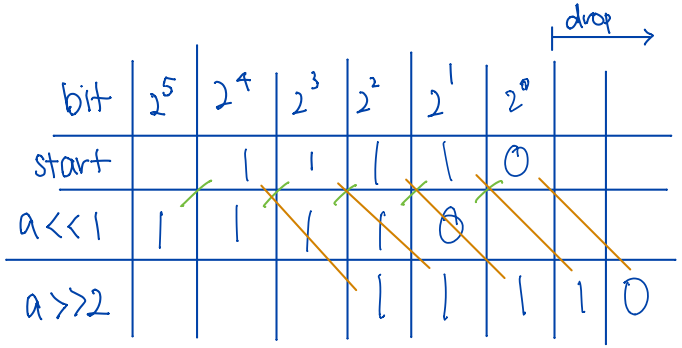
Bitwise Shift Operators

<< floor multiplication by 2/4/8 ($2^1, 2^2, 2^3$)

Operator	Description	Example
<<	**Moves the bits of its first operand to the left by the number of places specified in its second operand.**	**a << 1 -> 60**
>>	Pushes the bits of its first operand to the right by the number of places specified in its second operand.	a >> 1 -> 15

```
a = 30
>>> bin(a)
'0b11110'
>>> a << 1
60
>>> bin(60)
'0b111100'
```





$$0 \times 2^0 + 0 \times 2^1 + 1 \times 2^2 + 1 \times 2^3 + 1 \times 2^4 + 1 \times 2^5 = 60$$

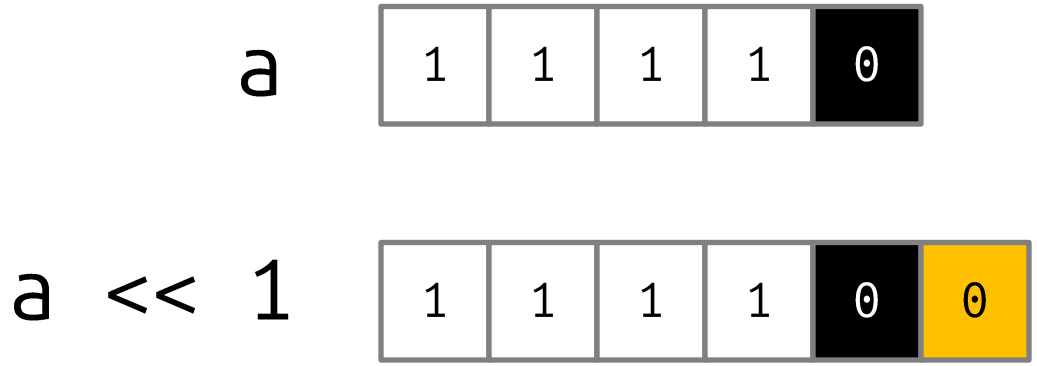
© MITB, SMU

Bitwise Shift Operators

>> floor division by 2/4/8 ($2^1, 2^2, 2^3$)

Operator	Description	Example
<<	**Moves the bits of its first operand to the left by the number of places specified in its second operand.**	**a << 1 -> 60**
>>	Pushes the bits of its first operand to the right by the number of places specified in its second operand.	a >> 1 -> 15

```
a = 30
>>> bin(a)
'0b11110'
>>> a << 1
60
>>> bin(60)
'0b111100'
```



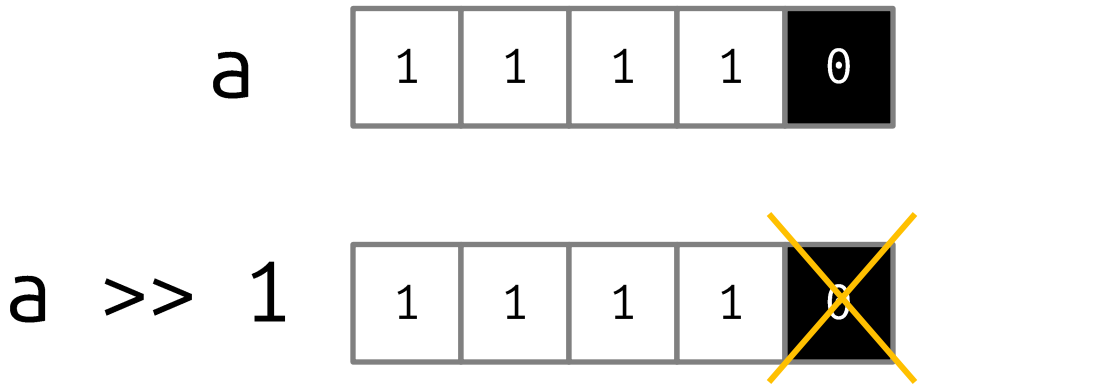
$$a \ll n \quad \Rightarrow \quad a \times 2^n$$

© MITB, SMU

Bitwise Shift Operators

Operator	Description	Example
<<	Moves the bits of its first operand to the left by the number of places specified in its second operand.	a << 1 -> 60
>>	**Pushes the bits of its first operand to the right by the number of places specified in its second operand.**	**a >> 1 -> 15**

```
a = 30
>>> bin(a)
'0b11110'
>>> a >> 1
15
>>> bin(15)
'0b1111'
```



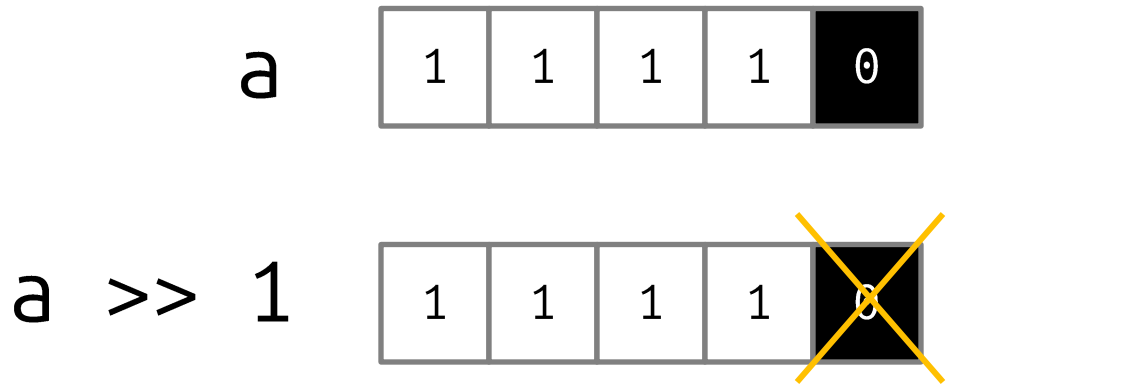
$$1 \times 2^0 + 1 \times 2^1 + 1 \times 2^2 + 1 \times 2^3 = 15$$

© MITB, SMU

Bitwise Shift Operators

Operator	Description	Example
<<	Moves the bits of its first operand to the left by the number of places specified in its second operand.	a << 1 -> 60
>>	**Pushes the bits of its first operand to the right by the number of places specified in its second operand.**	**a >> 1 -> 15**

```
a = 30
>>> bin(a)
'0b11110'
>>> a >> 1
15
>>> bin(15)
'0b1111'
```



$$a \gg n \quad \Rightarrow \quad \left\lfloor \frac{a}{2^n} \right\rfloor$$

© MITB, SMU

Wooclap!

What is the output of the below print statement?

```
a = 2
while a < 20:
    a = a << 1
print(a)
```

enters infinite loop @ 0

a. An infinite loop is observed
b. 20
c. 32
d. 64

Exercise 8-1: Binary Search – Bitwise


```
def BinarySearch(ls, target):
    min_idx, max_idx = 0, len(ls) - 1
    while min_idx <= max_idx:
        mid_idx = (min_idx + max_idx) // 2
        if ls[mid_idx] == target:
            return mid_idx
        elif ls[mid_idx] < target:
            min_idx = mid_idx + 1
        else:
            max_idx = mid_idx - 1
    return -1
```


© MITB, SMU

Exercise 8-2: Hire Chefs

To optimize our restaurant's dish offerings while managing costs, we need to hire two chefs with distinct skill sets. Given that hiring chefs with more diverse skills is more expensive, what combination of two chefs should we hire to maximize the number of unique dish combinations, and how many total combinations can be created?

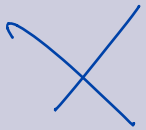
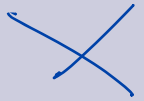
- Input

```
{'John': ['Apple Pie', 'Bak Kut Teh', 'Egg Prata'],
'Zach': ['Chilli Crab'],
'Amy': ['Bak Kut Teh', 'Duck rice', 'Chilli Crab'],
'Bobby': ['Duck rice', 'Fish Soup']}
```

- Output `('John', 'Bobby', 6)`

© MITB, SMU

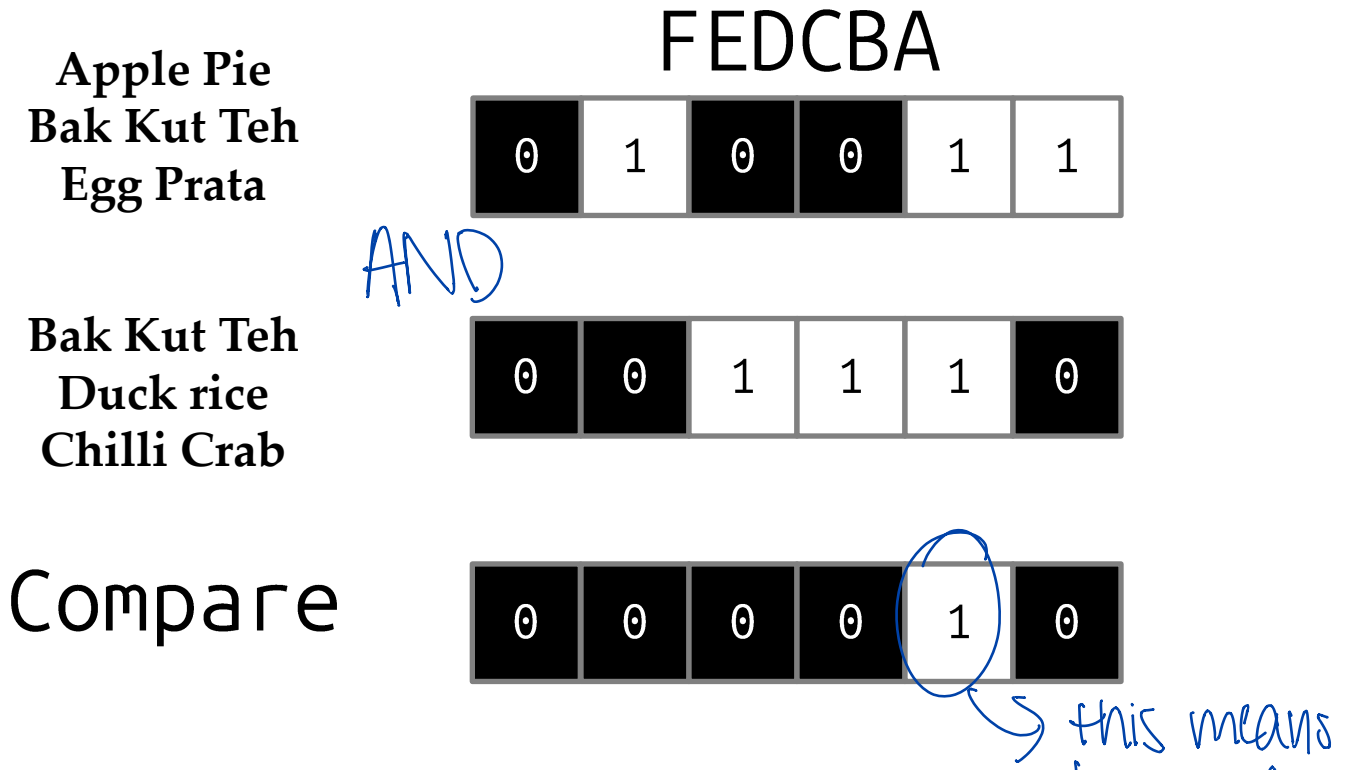
Exercise 8-2: Naive Solution

	Apple Pie Bak Kut Teh Egg Prata	Chilli Crab	Bak Kut Teh Duck rice Chilli Crab	Duck rice Fish Soup
Apple Pie Bak Kut Teh Egg Prata		3	Bak Kut Teh	6 AD AF BD BF ED EF
Chilli Crab			Chilli Crab	2
Bak Kut Teh Duck rice Chilli Crab				Duck rice
Duck rice Fish Soup				

© MITB, SMU

Exercise 8-2: Solution (Map to bit)

Dishes	Bit Masks
Apple Pie Bak Kut Teh Egg Prata	0b10011
Chilli Crab	0b100
Bak Kut Teh Duck rice Chilli Crab	0b1110
Duck rice Fish Soup	0b101000

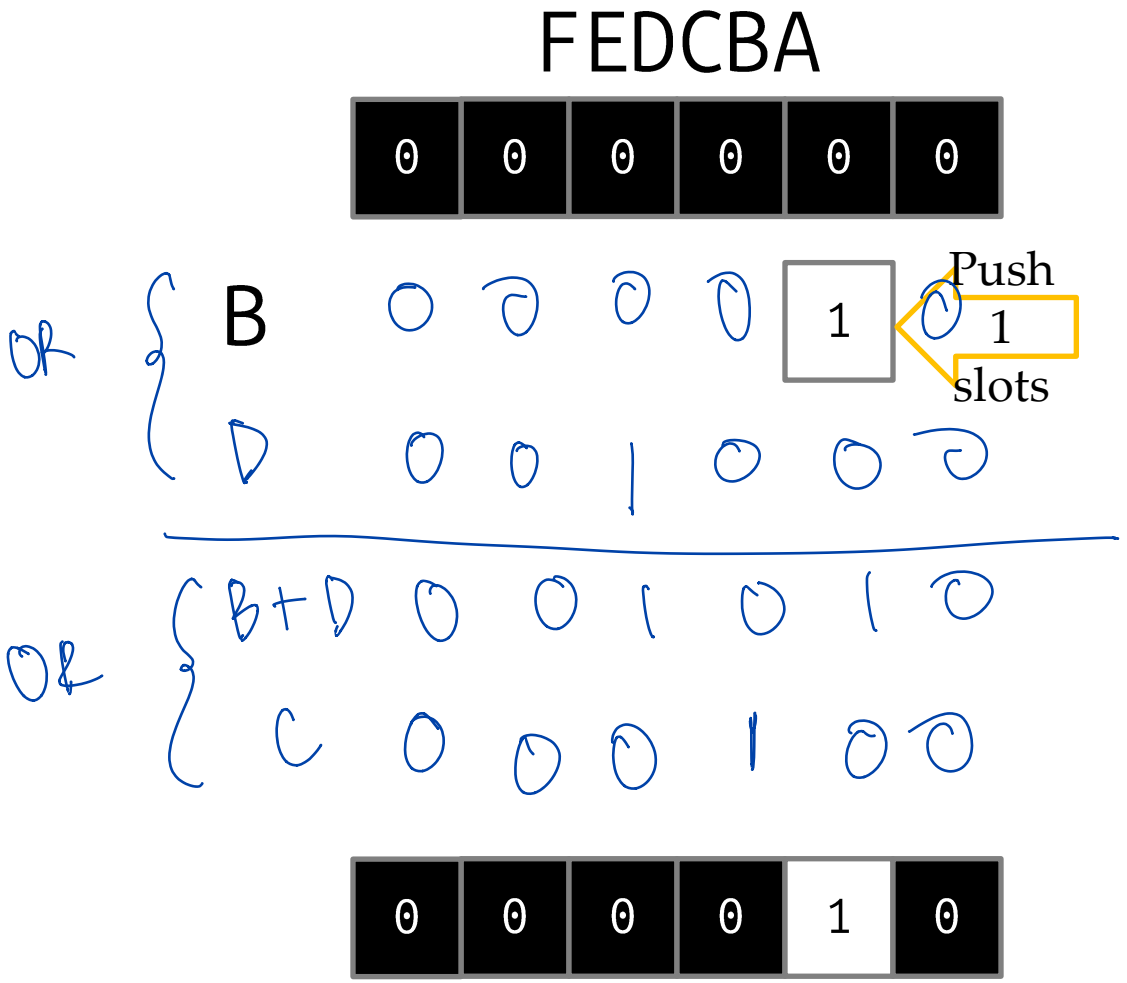


Which bitwise operator?

© MITB, SMU

Exercise 8-2: Solution (Map to bit)

Dishes	Bit Masks
Apple Pie Bak Kut Teh Egg Prata	0b10011
Chilli Crab	0b100
Bak Kut Teh Duck rice Chilli Crab	0b1110
Duck rice Fish Soup	0b101000

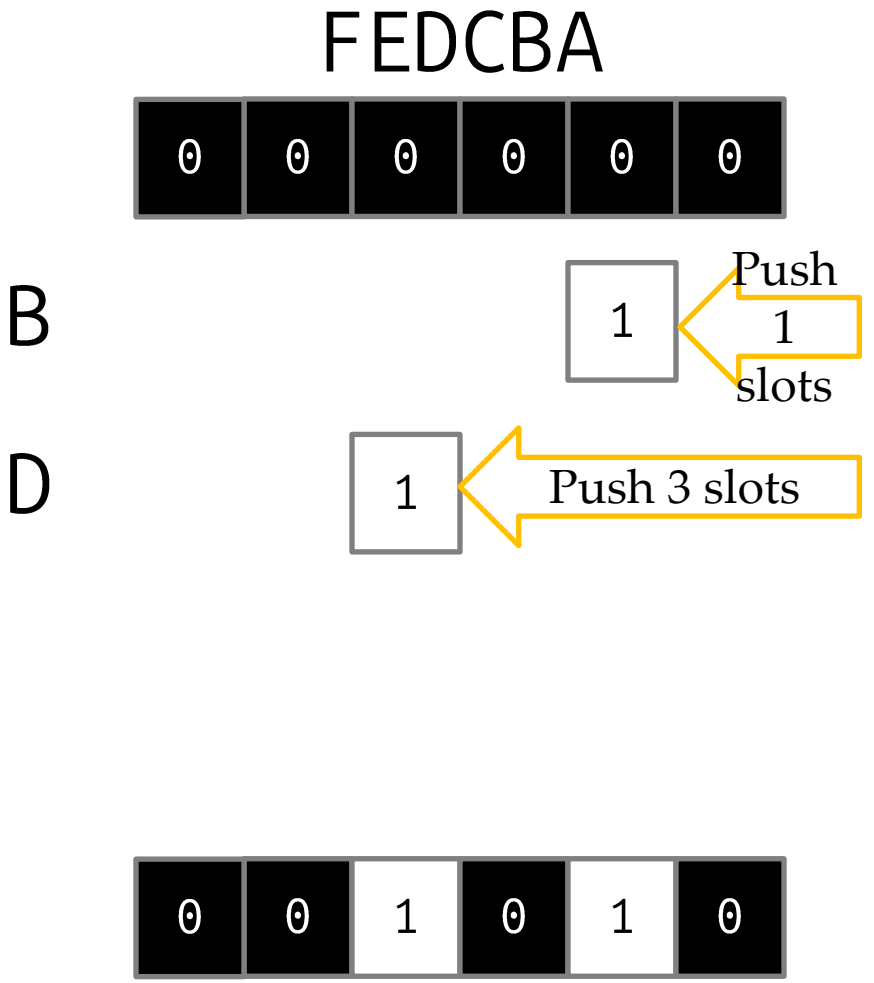


© MITB, SMU

Exercise 8-2: Solution (Map to bit)

Dishes	Bit Masks
Apple Pie Bak Kut Teh Egg Prata	0b10011
Chilli Crab	0b100
Bak Kut Teh Duck rice Chilli Crab	0b1110
Duck rice Fish Soup	0b101000

OR



© MITB, SMU

Exercise 8-2: Solution (Map to bit)

Dishes	Bit Masks
Apple Pie Bak Kut Teh Egg Prata	0b10011
Chilli Crab	0b100
Bak Kut Teh Duck rice Chilli Crab	0b1110
Duck rice Fish Soup	0b101000

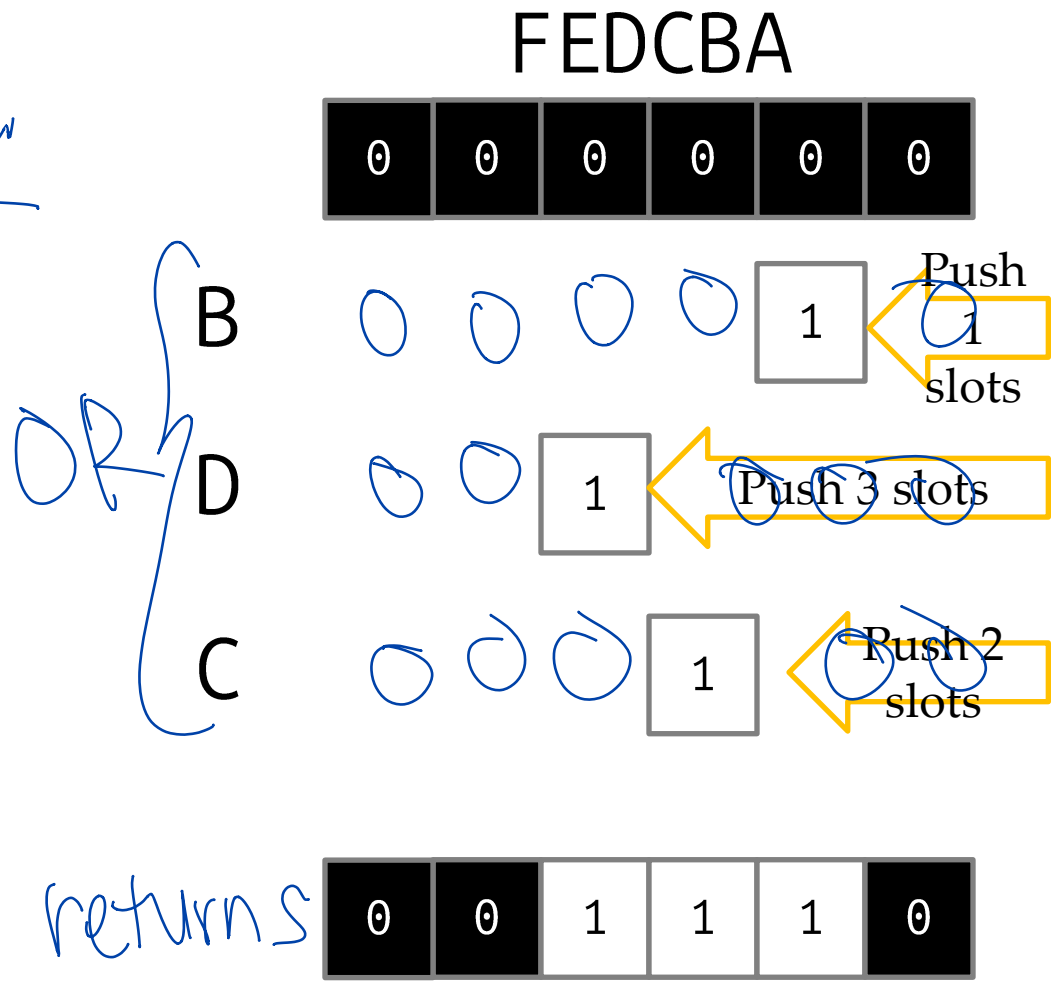
integer

19

4

= 14

= 40



Which bitwise operator?

© MITB, SMU

Hash Function

(Optional)

Why?

Optional

- Find Molly in your contact list
- Option 1: Go down the list one by one
- Option 2: Go to index M before searching
- Option 3: Ask Siri to get Molly's contact

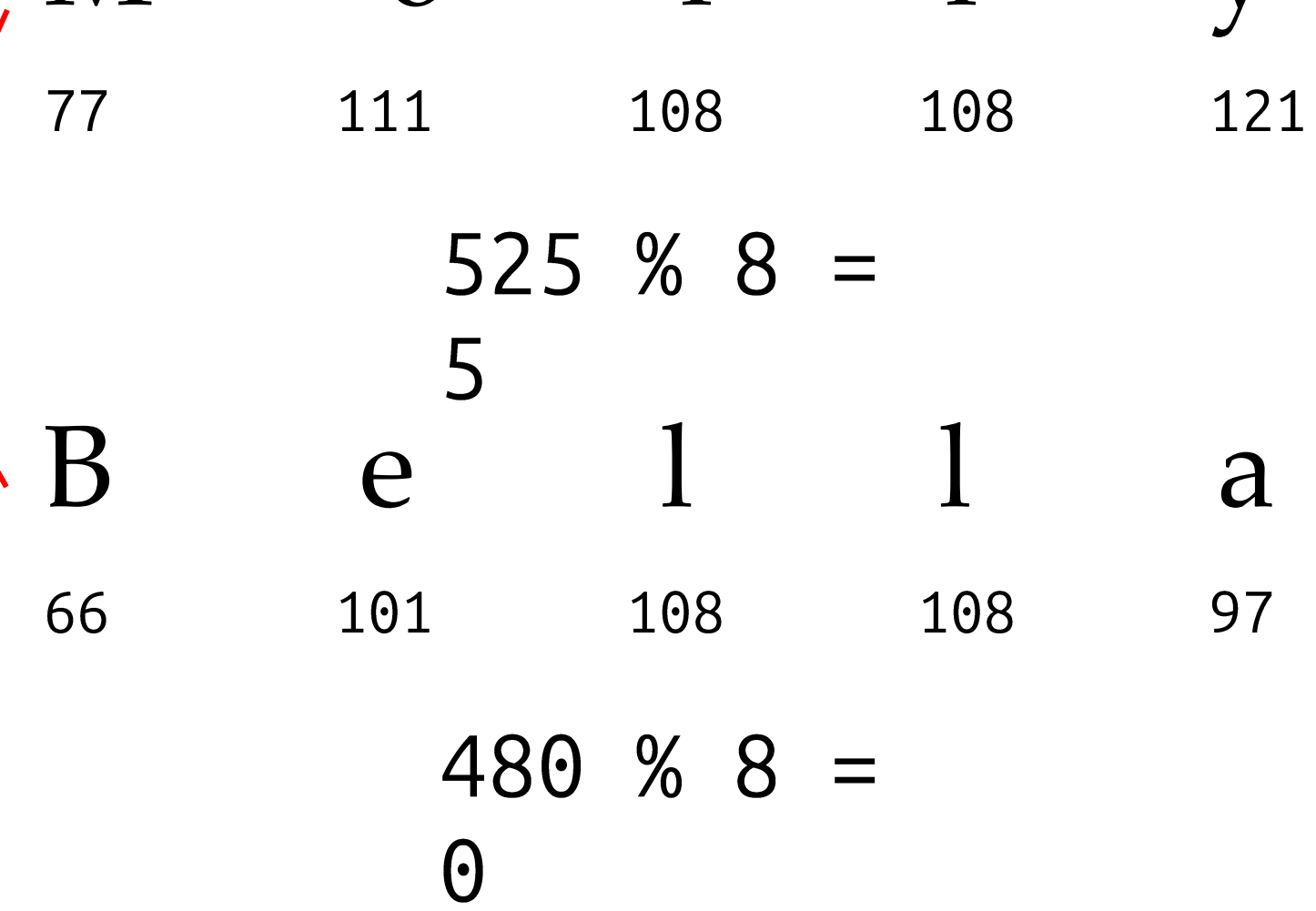
Fast data access

© MITB, SMU

How it works?

Optional

Address	Value
0	
1	
2	
3	
4	
5	
6	
7	



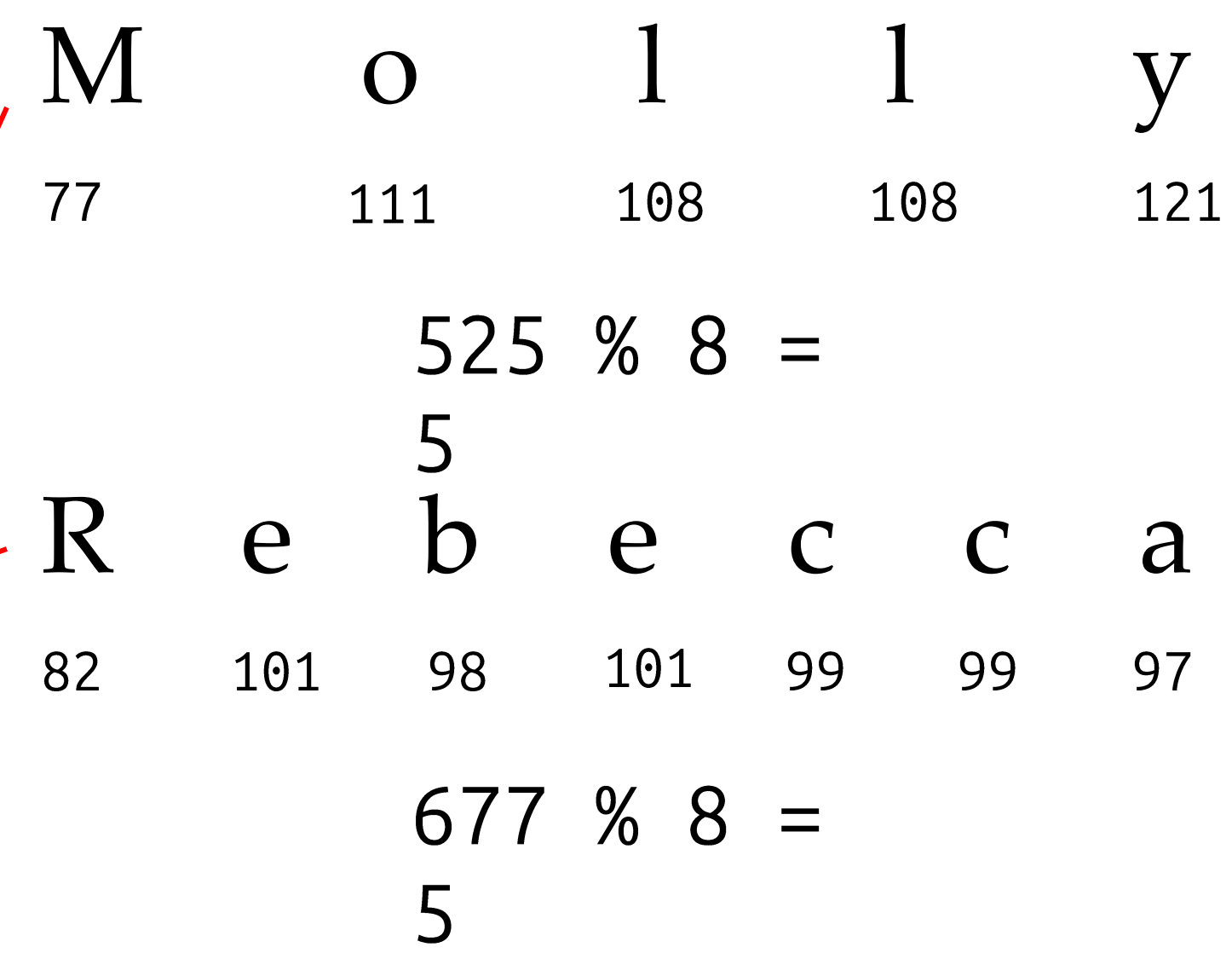
© MITB, SMU

Hash Collision

Optional

SMU Singapore Management University | School of Computing and Information Systems

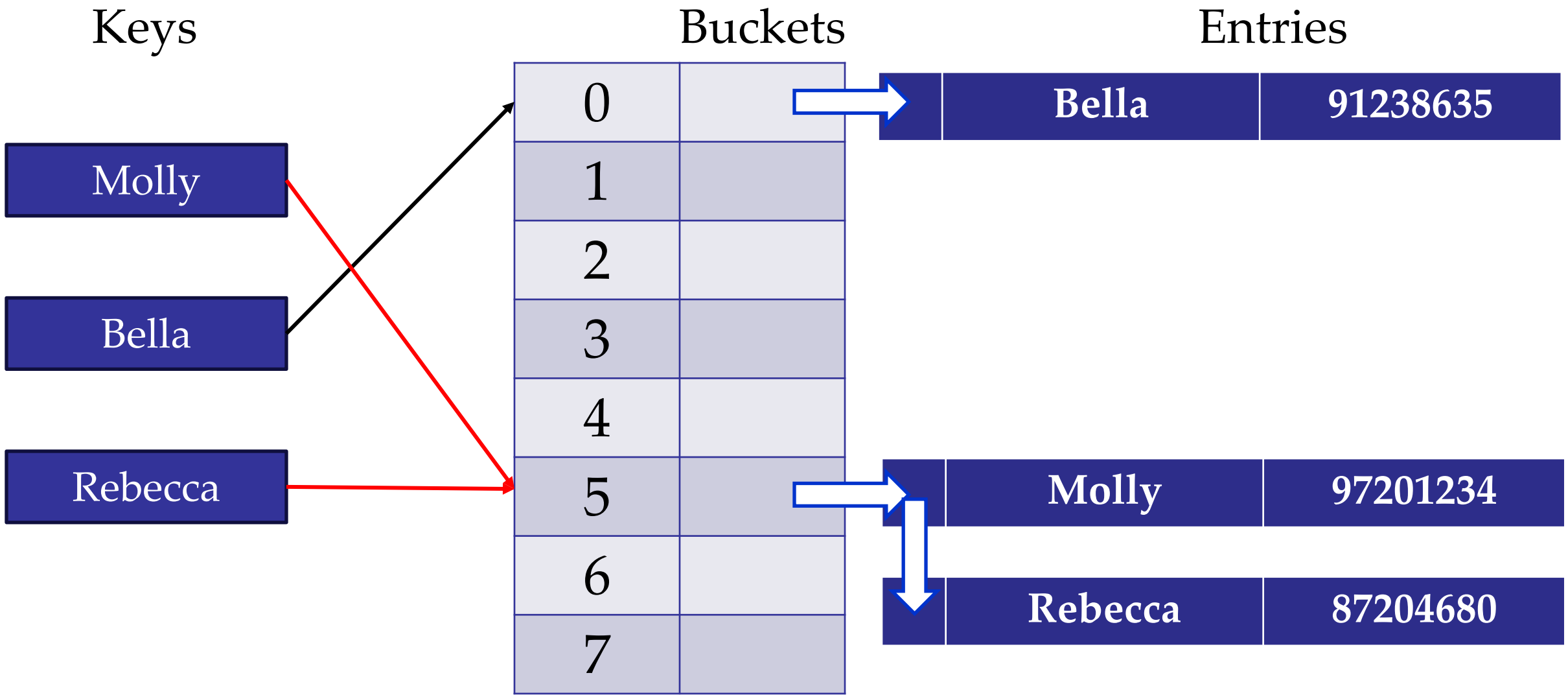
Address	Value
0	
1	
2	
3	
4	
5	
6	
7	



© MITB, SMU

Collision Resolution – Separate chaining

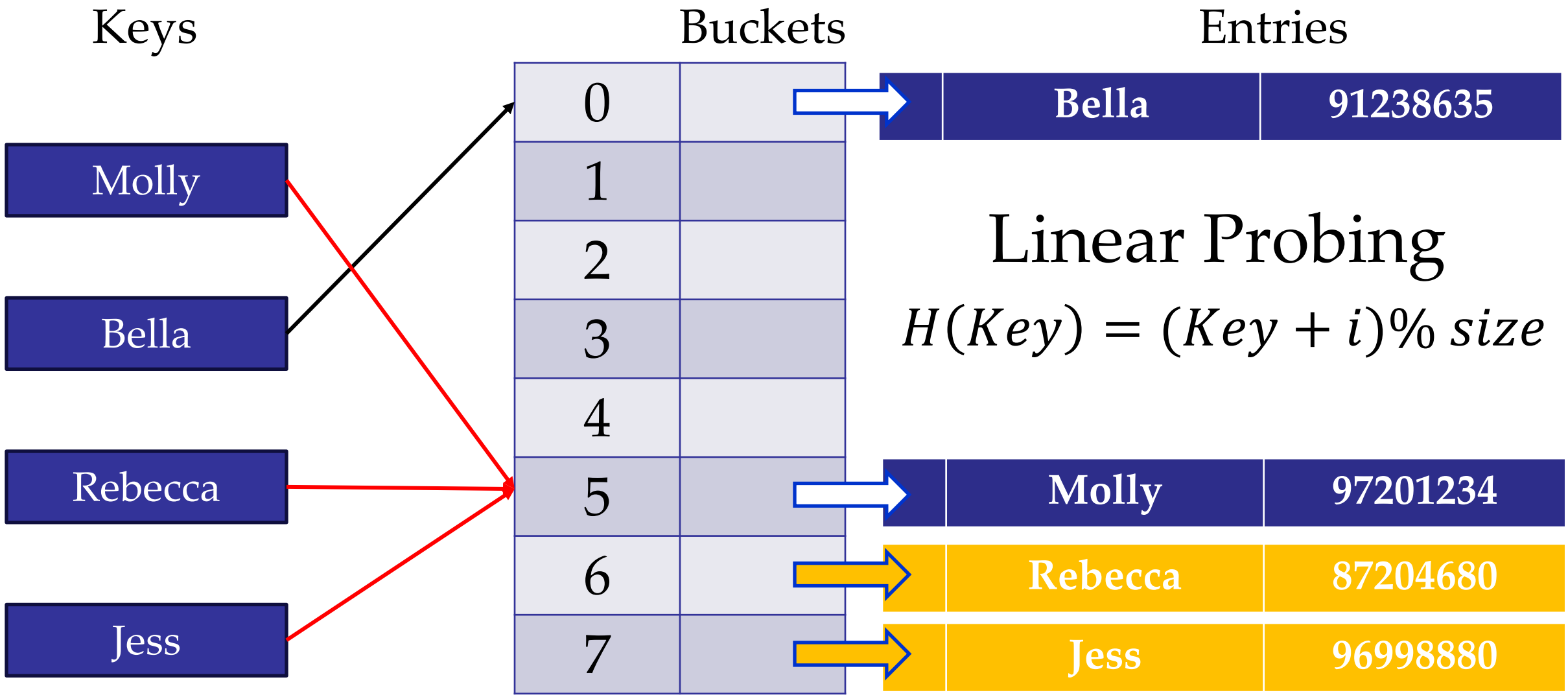
Optional



© MITB, SMU

Collision Resolution – Open addressing

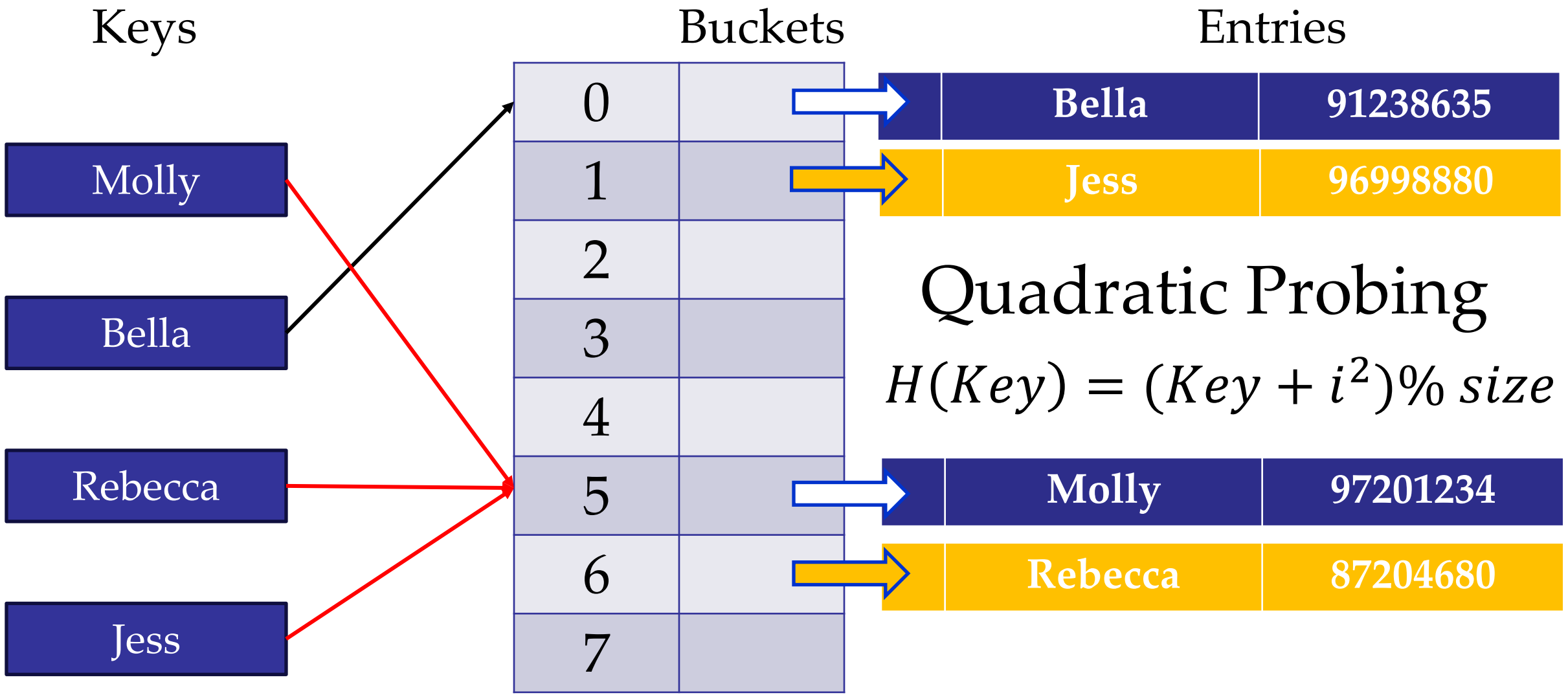
Optional



© MITB, SMU

Collision Resolution – Open addressing

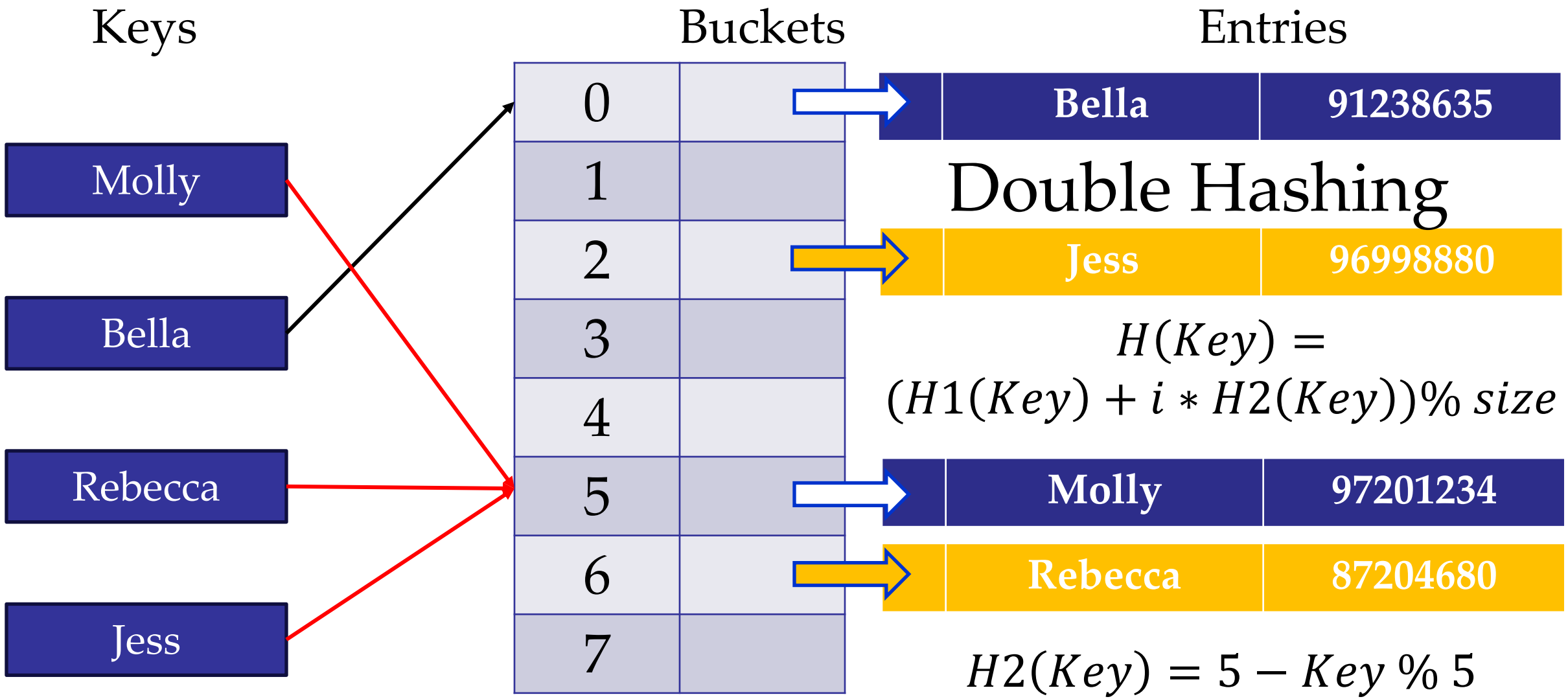
Optional



© MITB, SMU

Collision Resolution – Open addressing

Optional



© MITB, SMU

What is Hashable?

Optional

Hashable/ Immutable	Not Hashable/ Mutable
Strings	Lists
Integers	Dictionaries
Tuples	Arrays

© MITB, SMU

Other applications for hash functions

Optional

- If we do not want to handle hash collision:

Check existence, or check membership in a set: the "`in`" operator

- When a set is small ⇒ linear search
- Medium size? May consider binary search.
- How about a set with expected size 1,000,000?
  - hash table of 1,000,000 rows?

- Can we do better?
  - Can we build a table of 100 rows to check existence of 1,000,000 distinct items?

© MITB, SMU

Bloom Filter

Optional

Bloom Filter:

- identify something that is definitely not in a large set.

- Medium uses bloom filters for recommending post to users by filtering post which have been seen by user.
- Quora implemented a shared bloom filter in the feed backend to filter out stories that people have seen before.
- The Google Chrome web browser used to use a Bloom filter to identify malicious URLs

© MITB, SMU

Bloom Filter

Optional

Bloom Filter:

- identify something that is definitely not in a large set.

For example, to ensure there are no duplicate usernames.

- No False negative
  - If a username already exists and the system failed to detect it, an error occurs.
- Accept False positive
  - If a username doesn't exist but the system mistakenly indicates that it does, it's not a major issue.

		System Indicator	
		Present	Absent
Actually	Present	True Positive	False Negative
	Absent	False Positive	True Negative

© MITB, SMU

How it works?

Optional

SMU Singapore Management University | School of Computing and Information Systems

0	1	2	3	4	5	6	7	8	9
0	0	0	0	0	0	0	0	0	0

Name					
Hash1 % 10					
Hash2 % 10					

© MITB, SMU

How it works?

Optional

After Adding in Alice

0	1	2	3	4	5	6	7	8	9
0	1	0	0	0	1	0	0	0	0

- Add Alice in.

Name	Alice				
Hash1 % 10	1				
Hash2 % 10	5				

© MITB, SMU

How it works?

Optional

After Adding in Alice and Bob

0	1	2	3	4	5	6	7	8	9
0	1	1	1	0	1	0	0	0	0

- Check if Bob can be added in, if yes, add Bob in.

Name	Alice	Bob			
Hash1 % 10	1	2			
Hash2 % 10	5	3			

© MITB, SMU

How it works?

Optional

After Adding in Alice and Bob

0	1	2	3	4	5	6	7	8	9
0	1	1	1	0	1	0	0	0	0

- Check if Cindy can be added -> Yes, as both values at index 0 and 8 are 0.

Name	Alice	Bob	Cindy		
Hash1 % 10	1	2	0		
Hash2 % 10	5	3	8		

© MITB, SMU

How it works?

Optional

After Adding in Alice, Bob and Cindy

0	1	2	3	4	5	6	7	8	9
1	1	1	1	0	1	0	0	1	0

- Check if Daniel can be added -> Yes, as value at index 6 is 0.

Name	Alice	Bob	Cindy	Daniel	
Hash1 % 10	1	2	0	2	
Hash2 % 10	5	3	8	6	

© MITB, SMU

How it works?

Optional

After Adding in Alice, Bob, Cindy and Daniel

0	1	2	3	4	5	6	7	8	9
1	1	1	1	0	1	1	0	1	0

False Positive

- Check if Emma can be added -> No, as both values at index 3 and 5 are 1.

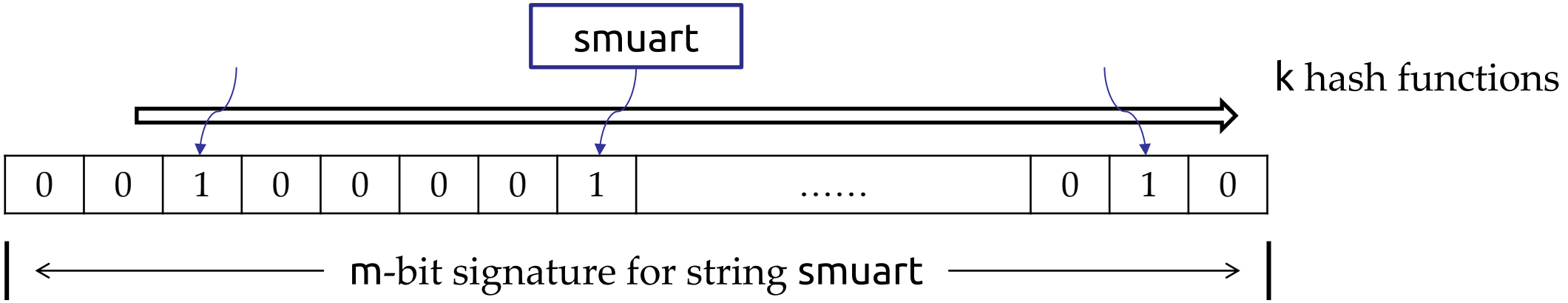
Name	Alice	Bob	Cindy	Daniel	Emma
Hash1 % 10	1	2	0	2	3
Hash2 % 10	5	3	8	6	5

© MITB, SMU

In a summary

Optional

- For each item, bloom filter creates a bitstring as the signature of this item:



$$\binom{128}{10} \approx 227,000,000,000,000$$

- Question 1: How many 128-bit signature with exactly 10 bits 1? m =128, k = 10
- Question 2: Given a set with the expected size 1,000,000, how likely a false positive will occur?

We need a much larger m for a tiny probability of false positivity

© MITB, SMU

Probability of False Positivity

Optional

- A bit is 0 after adding n items:
  - None of the n items, none of the k hash functions have ever touched this bit

$$(1 - \frac{1}{m})^{kn} \approx e^{-\frac{kn}{m}}$$

Lecture 5 on Limits

0.9282

- A bit is 1 after adding n items:

$$1 - (1 - \frac{1}{m})^{kn} \approx 1 - e^{-\frac{kn}{m}}$$

0.0718

- A new item happens to have hash values 1 at all k positions:

$$[1 - (1 - \frac{1}{m})^{kn}]^k \approx [1 - e^{-\frac{kn}{m}}]^k$$

3.64e-12

134,217,728

m	Number of bits
	e.g., m = 16MB = 2^{27} bits
k	Number of hash functions
	e.g., k = 10
n	Number of items
	e.g., n = 1,000,000

© MITB, SMU

Exercise 8-3: Bloom Filter

Optional

- Write a class BloomFliter to check if a username already exists.
  - Attribute `size`: size of the bitarray
  - Attribute `num_hash`: the number of hash functions
  - Method `add(username)`: Hash the username and store in the bit array
  - Method `check(username)`: Return `True` if the username might exist
- Try out MurmurHash https://pypi.org/project/mmh3/ for hash functions.
- Use `all()`

© MITB, SMU

Questions?

Thank You

